

Лабораторная работа №5. Jetpack Compose: контейнеры, ARGB-модель и Material Design

Цель работы: Повторить основы разработки пользовательского интерфейса на **Jetpack Compose**, работу с контейнерами **Row** и **Column**, познакомиться с **ARGB-моделью** цветов, компонентом **Scaffold** и принципами **Material Design**.

Шаг 1. Создание проекта

- Откройте **Android Studio**
- Выберите **File** → **New** → **New Project**
- В окне выбора шаблона выберите **Empty Activity** → **Next**
- Укажите параметры:
 - **Name:** **Android_Lab_5_FIO** (замените FIO на вашу фамилию)
 - **Minimum SDK:** API 25
- Нажмите **Finish**

Empty Activity

Create a new empty activity with Jetpack Compose

Name: **Android_Lab_5_Leontev**

Package name: **com.leontev.android_lab_5_leontev**

Save location: **/home/denis/AndroidStudioProjects/Android_Lab_5_Leontev**

Minimum SDK: **API 25 ("Nougat"; Android 7.1.1)**

Build configuration language: **Kotlin DSL (build.gradle.kts) [Recommended]**

Buttons: Previous, Next, Cancel, **Finish**

Дождитесь полной загрузки проекта.

Шаг 2. Основы MainActivity и Composable-функций

2.1. Очистка стартового шаблона

В файле **MainActivity.kt** удалите лишние **@Composable-функции**, оставив базовую структуру активности. Оптимизируйте импорты, нажав **Ctrl+Alt+O**

```

package com.leontev.android_lab_5_leontev

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge

1 Usage
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {

        }
    }
}

```

Пояснение к коду:

- **package** — указывает пакет приложения, уникальный идентификатор вашего проекта
- **import** — подключает необходимые классы и функции из библиотек Android
- **class MainActivity : ComponentActivity()** — главный класс активности, наследуется от ComponentActivity для поддержки Jetpack Compose
- **override fun onCreate()** — метод жизненного цикла, вызывается при создании активности
- **super.onCreate(savedInstanceState)** — вызов метода родительского класса для корректной инициализации
- **enableEdgeToEdge()** — включает режим отображения от края до края экрана (использует всю площадь, включая область системных панелей)
- **setContent { }** — функция Compose для определения UI приложения в декларативном стиле

2.2. Создание первого UI-элемента

Добавим текстовый элемент внутри setContent {}:

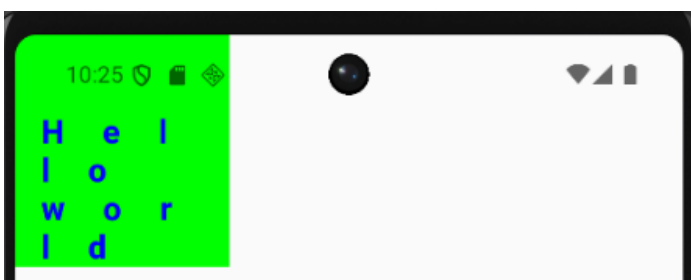
```
setContent {
    Text(
        modifier = Modifier
            .background(color = Color.Green)
            .padding(top = 48.dp, start = 16.dp, end = 16.dp)
            .width(100.dp),
        text = "Hello world",
        fontWeight = FontWeight.ExtraBold,
        color = Color.Blue,
        fontSize = 20.sp,
        letterSpacing = 24.sp
    )
}
```

Пояснение:

- **Text()** — Composable-функция для отображения текста
- **modifier** — модификатор для настройки внешнего вида и поведения компонента
- **.background()** — устанавливает цвет фона (зелёный)
- **.padding()** — задаёт внутренние отступы (48dp сверху, 16dp слева и справа)
- **.width()** — ограничивает ширину элемента до 100dp
- **text** — отображаемый текст
- **fontWeight** — толщина шрифта (ExtraBold — очень жирный)
- **color** — цвет текста (синий)
- **fontSize** — размер шрифта в sp (масштабируемых пикселях)
- **letterSpacing** — межбуквенный интервал

При добавлении этого кода Android Studio предложит импортировать недостающие классы. Согласитесь, нажав **Alt+Enter** и выбрав **"Import"**.

Запустите приложение на эмуляторе или устройстве (**Shift+F10**) и убедитесь, что текст отображается с зелёным фоном:



Шаг 3. Контейнеры Row и Column

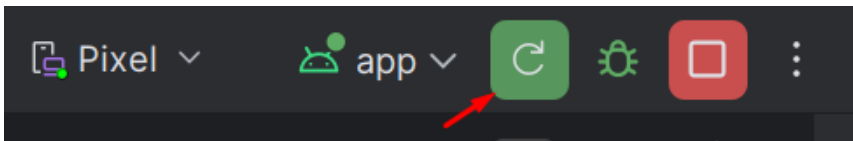
3.1. Проблема наложения элементов

Попробуем вывести текст несколько раз с помощью цикла **repeat()**:

```
setContent {
  repeat(5) {
    Text(...)
  }
}
```



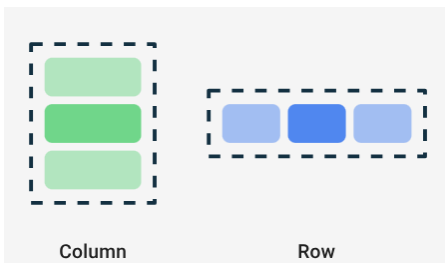
Перезапустите приложение:



Результат: на экране по-прежнему виден только один текстовый элемент. Причина — все пять элементов **Text()** отрисовываются в одной точке, накладываясь друг на друга.

Чтобы управлять расположением элементов, используются контейнеры:

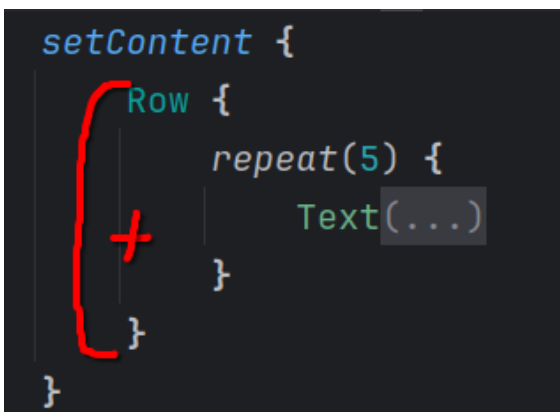
- **Row { }** — располагает элементы в **строку** (горизонтально, слева направо);
- **Column { }** — располагает элементы в **столбик** (вертикально, сверху вниз).



3.2. Использование Row (горизонтальное размещение)

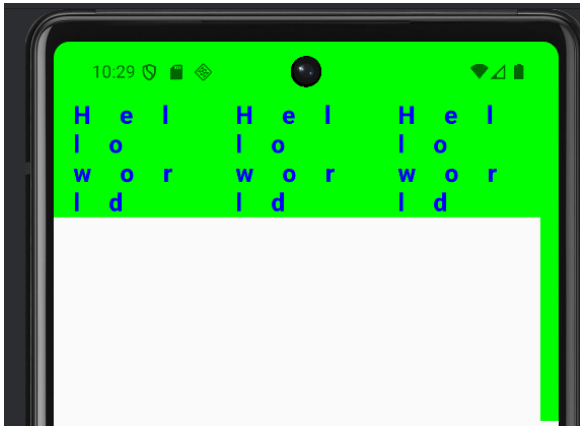
Оберните вызов **repeat** в контейнер **Row**:

```
setContent {
  Row {
    repeat(5) {
      Text(...)
    }
  }
}
```



Перезапустите приложение.

Результат: текстовые блоки расположены горизонтально. Часть из них не помещается на экране и остаётся невидимой:



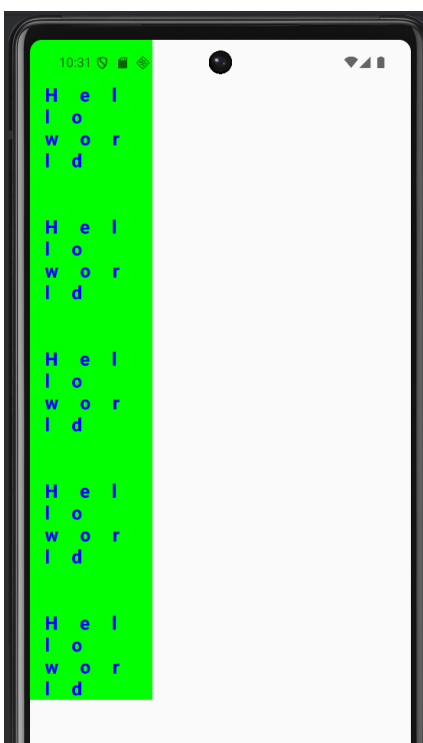
3.3. Использование Column (вертикальное размещение)

Замените **Row** на **Column**:

```
setContent {  
    Column {  
        repeat(5) {  
            Text(...)   
        }  
    }  
}
```

Перезапустите приложение.

Результат: текстовые блоки выстроены вертикально. Если элементов много, часть из них выйдет за пределы экрана:



Шаг 4. Посимвольный вывод строки

Теперь реализуем отображение каждого символа строки как отдельного элемента `Text`:

- Отфильтруем строку так, чтобы остались только буквы.
- Переберём все символы полученной строки в цикле и для каждого создадим отдельный `Text()`.
- Выполним всё это внутри контейнера `Row { }`.
- Добавим необходимые отступы для удобства восприятия.

4.1. Подготовка текста

Создадим переменную для текста и отфильтруем только буквы:

```
setContent {  
    val symbols = "Hello world!".filter { it.isLetter() }
```

Пояснение:

- `val text = ...` — создаём неизменяемую переменную
- `.filter { it.isLetter() }` — фильтруем строку, оставляя только буквы (удаляются пробелы, цифры, знаки препинания)
- Результат: "HelloWorld"

4.2. Вывод каждого символа отдельно

Заменяем `repeat` на цикл `for`, чтобы пройтись по каждому символу:

```
Column {  
    for(char in symbols) {  
        Text(  
            text = char.toString(),
```

Обратите внимание: внутри `Text()` символ необходимо привести к строке с помощью `char.toString()`, иначе **Kotlin** не сможет корректно интерпретировать одиночный символ как текст:

```
text = char.toString(),
```

4.3. Обновляем модификаторы

Внесём изменения в код:

- Убран параметр `width` — теперь ширина определяется содержимым
- `padding(8.dp)` — упрощённый отступ со всех сторон

- **background()** перенесён после **padding()** — логически фон применяется после отступов

- Переносим модификаторы в **Column**:

```
setContent {  
    val symbols = "Hello world!".filter { it.isLetter() }  
    Column(  
        modifier = Modifier  
            .padding(8.dp)  
            .background(color = Color.Green),  
    ) {  
        for(char in symbols) {  
            Text(  
                text = char.toString(),  

```

Шаг 5. Управление размером и расположением контейнеров

5.1. Модификаторы размера

Jetpack Compose предоставляет модификаторы для управления размером контейнера:

- **.fillMaxWidth()** — контейнер занимает всю доступную ширину экрана
- **.fillMaxHeight()** — контейнер занимает всю доступную высоту экрана
- **.fillMaxSize()** — контейнер растягивается на всё доступное пространство

Попробуйте добавить **.fillMaxSize()** к **Column**:

```
Column(  
    modifier = Modifier  
        + .fillMaxSize()  
        .padding(8.dp)  
        .background(color = Color.Green),  
) {
```

5.2. Горизонтальное выравнивание (Arrangement)

Параметр **horizontalArrangement** (для **Row**) или **verticalArrangement** (для **Column**) управляет распределением элементов внутри контейнера.

Центрирование элементов

```
Column(  
    modifier = Modifier  
        .fillMaxSize()  
        .padding(8.dp)  
        .background(color = Color.Green),  
    horizontalAlignment = Alignment.CenterHorizontally  
) {
```

Результат: все символы центрированы по горизонтали и вертикали.

5.3. Варианты Arrangement

Попробуйте разные значения **verticalArrangement**:

1. **Arrangement.SpaceAround** — равные отступы до, между и после элементов

verticalArrangement = Arrangement.SpaceAround

2. **Arrangement.SpaceBetween** — элементы равномерно распределены, первый

прижат к верху, последний — к низу

verticalArrangement = Arrangement.SpaceBetween

3. **Arrangement.SpaceEvenly** — одинаковое расстояние между всеми элементами,

включая начало и конец

verticalArrangement = Arrangement.SpaceEvenly

```
Column(  
    modifier = Modifier  
        .fillMaxSize()  
        .padding(8.dp)  
        .background(color = Color.Green),  
    + verticalArrangement = Arrangement.SpaceEvenly,  
    horizontalAlignment = Alignment.CenterHorizontally  
) {
```

Задание: поочерёдно примените каждое значение, перезапустите приложение и изучите различия в расположении.

Шаг 6. Комбинированные контейнеры

Создание двух строк символов

Прежде чем начнём писать дальше, на текущий момент у вас код должен выглядеть примерно следующим образом:

```
24 <> class MainActivity : AppCompatActivity() {  
25 @< class MainActivity : AppCompatActivity() {  
26     override fun onCreate(savedInstanceState: Bundle?) {  
27         super.onCreate(savedInstanceState)  
28         enableEdgeToEdge()  
29         setContent {  
30             val symbols = "Hello world!".filter { it.isLetter() }  
31             Column(...) {  
32                 for(char in symbols) {  
33                     Text(...) }  
34             }  
35         }  
36     }  
37 }  
38 }  
39 }  
40 }  
41 }  
42 }  
43 }  
44 }  
45 }  
46 }  
47 }  
48 }  
49 }  
50 }
```


1. Добавим первую строку:

```
26      setContent {
27          val symbols = "Hello world!".filter { it.isLetter() }
28          Column(...) { 1 СТРОКА
36              Row(
37                  modifier = Modifier.fillMaxWidth(),
38                  horizontalArrangement = Arrangement.SpaceEvenly
39              ) {
40
41              }
42          for (char in symbols) {...}
51      }
52  }
53
54 }
```

2. Перенесём цикл **for** внутрь:

```
36      Row(
37          modifier = Modifier.fillMaxWidth(),
38          horizontalArrangement = Arrangement.SpaceEvenly
39      ) {
40          for (char in symbols) {...}
49      }
50
51 }
```

3. Добавим отступ:

```
38      Row(
39          modifier = Modifier.fillMaxWidth(),
40          horizontalArrangement = Arrangement.SpaceEvenly
41      ) {
42          for (char in symbols) {...}
51      }
52      + Spacer(modifier = Modifier.height(16.dp))
53 }
```

4. Теперь добавляем вторую строку, скопировав первый **Row** с содержимым:

```
30      Column(...) {
38          Row(...) {
42              for (char in symbols) {...}
51          }
52          Spacer(modifier = Modifier.height(16.dp))
53          Row(...) {
57              for (char in symbols) {...}
66          }
67      }
```

5. Для второй строки сделаем **реверс**, и поменяем **цвет**:

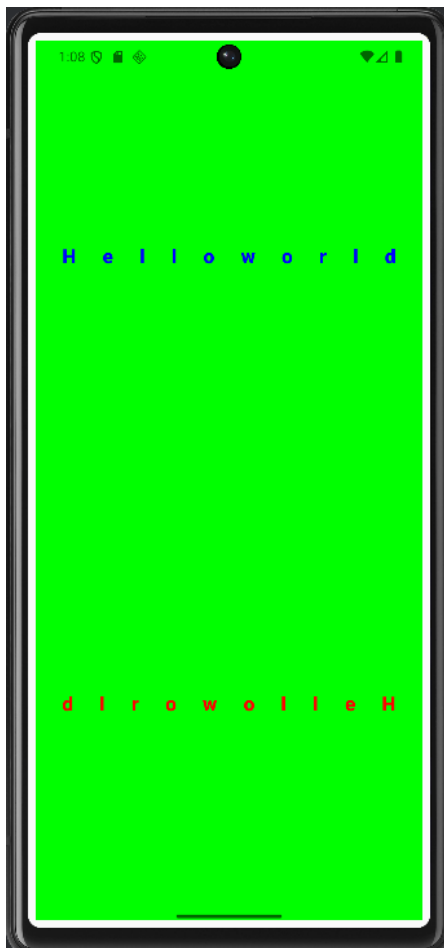
```
53      > Row(...) {
57          for (char in symbols.reversed()) {
58              Text(
59                  text = char.toString(),
60                  fontWeight = FontWeight.ExtraBold,
61                  color = Color.Red,
62                  fontSize = 20.sp,
63                  letterSpacing = 24.sp
```

Пояснение:

- Внешний **Column** — основной контейнер
- Два вложенных **Row** — каждая строка символов
- **Spacer(modifier = Modifier.height(16.dp))** — создаёт вертикальный отступ между строками
- **text.reversed()** — переворачивает строку задом наперёд
- Разные цвета для визуального различия строк

Перезапустите приложение.

Результат: две строки символов, расположенные вертикально, с разными цветами:



Шаг 7. ARGB-модель цветов

7.1. Что такое ARGB?

ARGB — модель представления цвета, где:

- **A** (Alpha) — прозрачность (0-255, где 0 = полностью прозрачный, 255 = непрозрачный)
- **R** (Red) — красный канал (0-255)
- **G** (Green) — зелёный канал (0-255)
- **B** (Blue) — синий канал (0-255)

7.2. Создание цветов в Compose

В Jetpack Compose цвета можно задавать несколькими способами:

1. Именованные цвета

- `Color.Red`
- `Color.Green`
- `Color.Blue`

2. Нех-формат (ARGB)

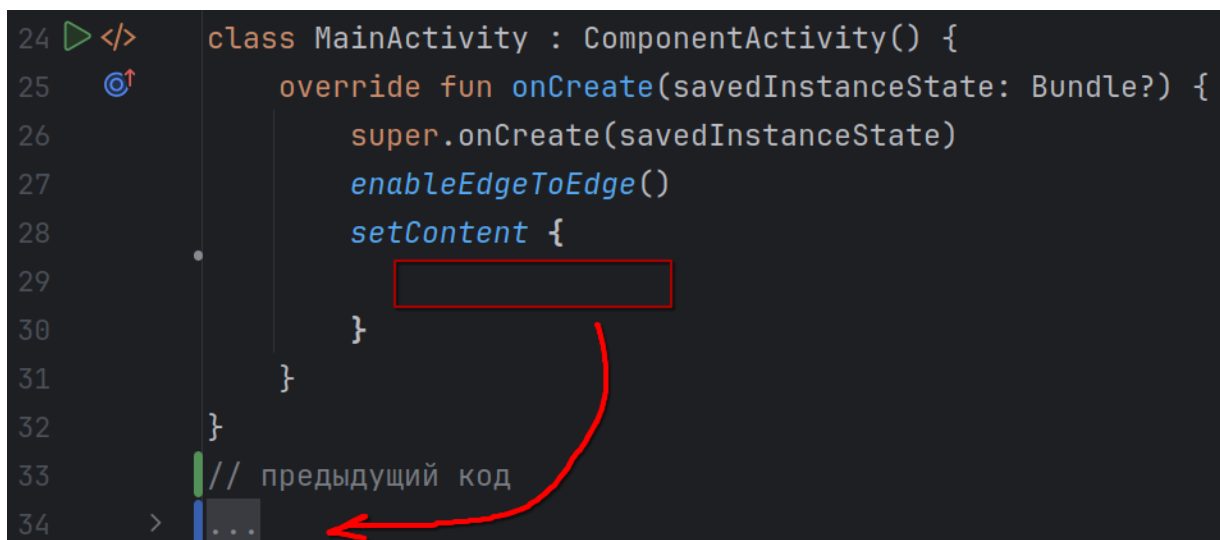
- `Color(0xFFFF0000)` // Красный (A=FF, R=FF, G=00, B=00)
- `Color(0x80FF0000)` // Полупрозрачный красный (A=80)

3. ARGB в десятичном формате

- `Color(255, 0, 0, 255)` // Красный (R, G, B, A)
- `Color(255, 0, 0, 128)` // Полупрозрачный красный

7.3. Практическая работа с ARGB

Вынесите за пределы **MainActivity** из **setContent** написанный ранее код и закомментируйте его, чтобы он не мешал текущему примеру.



```
24 </> class MainActivity : AppCompatActivity() {
25     override fun onCreate(savedInstanceState: Bundle?) {
26         super.onCreate(savedInstanceState)
27         enableEdgeToEdge()
28         setContent {
29             // ...
30         }
31     }
32 }
33 // предыдущий код
34 ...
```

1. Пропишем **Column** внутри **setContent**:

```
setContent {  
    Column(  
        modifier = Modifier  
            .fillMaxSize()  
            .background(Color.White)  
            .padding(16.dp),  
        verticalArrangement = Arrangement.SpaceEvenly,  
        horizontalAlignment = Alignment.CenterHorizontally  
    ) {  
    }  
}
```

Описание: На этом шаге мы создаём вертикальный контейнер **Column**, который занимает весь экран. Задаём белый фон, внутренние отступы и выравнивание элементов по центру. **Column** будет служить основой для размещения цветных блоков на экране.

2. Добавим внутрь непрозрачный красный:

```
Column(...) {  
    Box(  
        modifier = Modifier  
            .size(100.dp)  
            .background(Color(0xFFFF0000))  
    )  
}
```

Соглашаемся на импорты. Перезапускаем эмулятор и видим красный квадрат:



Описание: Здесь мы добавляем **Box** размером 100×100 dp и задаём ему цвет в ARGB-формате. Значение `0xFFFF0000` означает полностью непрозрачный красный цвет, так как альфа-канал равен FF (255).

3. Добавим полупрозрачный красный:

```
Column(...)  
{  
  Box(...)  
  Box(  
    modifier = Modifier  
      .size(100.dp)  
      .background(Color(0x80FF0000))  
  )  
}
```

Перезапускаем эмулятор и видим полупрозрачный красный квадрат:



Описание: В этом примере используется тот же красный цвет, но с альфа-каналом 80, что делает его полупрозрачным. Это позволяет наглядно увидеть, как параметр Alpha влияет на прозрачность элемента в интерфейсе.

Шаг 8. Scaffold и пользовательские компоненты

8.1. Что такое Scaffold?

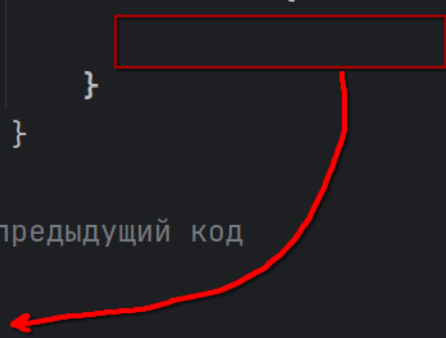
Scaffold — это готовый контейнер из Material Design, который помогает быстро собрать стандартный экран приложения. Он предоставляет заранее определённые области (слоты) для основных элементов интерфейса:

- **AppBar** — верхняя панель приложения
- **BottomBar** — нижняя панель навигации
- **FloatingActionButton** — плавающая кнопка действия
- **Drawer** — боковое выезжающее меню
- **Content** — основное содержимое экрана

Использование Scaffold позволяет соблюдать Material-гайды и упрощает структуру UI.

Вынесите за пределы **MainActivity** из **setContent** написанный ранее код и закомментируйте его, чтобы он не мешал текущему примеру.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            
        }
    }
}
// предыдущий код
...
...
```

A red box highlights a line of code inside the setContent block. A red arrow originates from this box and points to a line of code (commented out) located below the MainActivity class definition.

8.2. Создание экрана с Scaffold

1. Создадим простой экран с Scaffold внутри setContent:

```
setContent {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text(text = "Scaffold example") }
            )
        }
    ) { paddingValues ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(paddingValues),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(text = "Основное содержимое экрана")
        }
    }
}
```

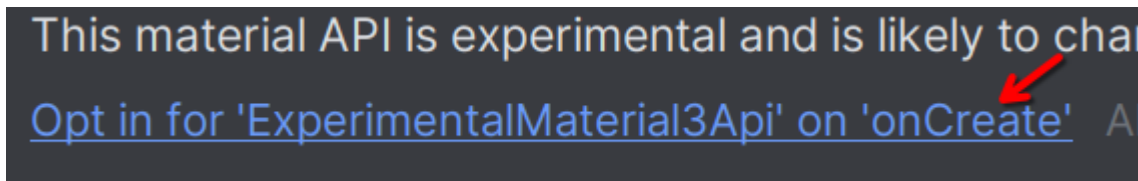
A red bracket on the left side of the code block groups the entire setContent function. A red plus sign is placed next to the Scaffold function call.

Для **TopAppBar** у нас возникло предупреждение:

This material API is experimental and is likely to change or to be removed in the future.

Opt in for 'ExperimentalMaterial3Api' on 'onCreate' Alt+Shift+Enter More actions... Alt+Enter

В Material 3 (material3.TopAppBar) многие компоненты помечены как experimental, поэтому компилятор честно предупреждает. Код рабочий, но нужно явно согласиться с использованием экспериментального API:



Описание: На этом шаге мы создаём экран с использованием Scaffold. В topBar добавляется верхняя панель с заголовком. Основное содержимое располагается внутри Content-слота, представленного Column. paddingValues используются для того, чтобы контент не перекрывался TopAppBar.

2. Добавим FloatingActionButton:



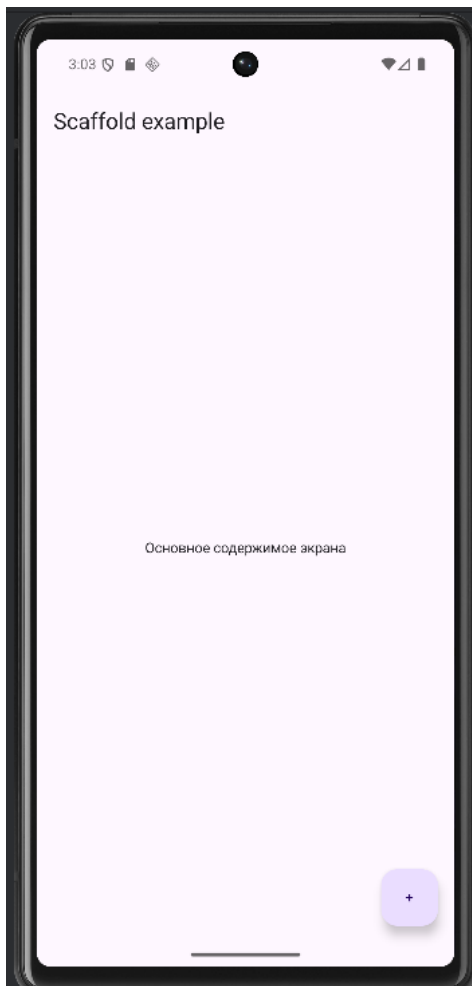
Описание:

Здесь мы добавляем плавающую кнопку действия (FloatingActionButton). Она автоматически размещается поверх контента в правом нижнем углу экрана, что соответствует стандартам Material Design.

3. Результат:

После запуска приложения мы видим:

- верхнюю панель с заголовком,
- текст в центре экрана,
- плавающую кнопку внизу справа.



Такой экран является базой для большинства приложений и может легко расширяться нижней навигацией, боковым меню и пользовательскими компонентами.

Самостоятельное задание

Задание:

1. Создать простое приложение на Jetpack Compose.
2. Реализовать главный экран с использованием:
 - `Column` как основного контейнера
 - `Row` для горизонтального расположения элементов
 - `Box` для отображения цветных блоков
3. На экране должны присутствовать:
 - заголовок из двух строк текста
 - горизонтальный ряд из **трёх цветных квадратов**
 - один квадрат должен быть **полупрозрачным**
 - один контейнер с вложенными `Row` и `Column`
4. Цвета задать в **ARGB-формате**.
5. Код экрана должен быть разбит на **отдельные @Composable функции**.

Пример реализации:

```
class MainActivity : ComponentActivity() {
    @OptIn(ExperimentalMaterial3Api::class)
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MainScreen()
        }
    }
}

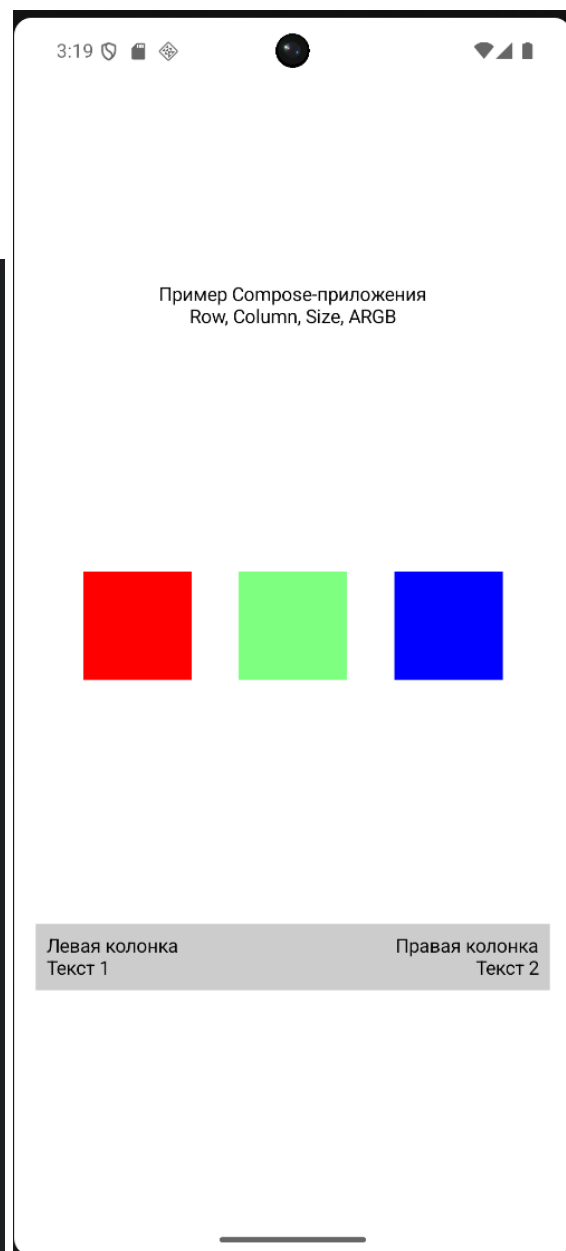
1 Usage
@Composable
fun TitleBlock() {...}

1 Usage
@Composable
fun MainScreen() {...}

1 Usage
@Composable
fun ColorRow() {...}

3 Usages
@Composable
fun ColorBox(color: Color) {...}

1 Usage
@Composable
fun CombinedBlock() {...}
```



Работа с GitHub

1. Создать публичный репозиторий на GitHub.
2. Загрузить в репозиторий проект Android Studio.
3. Оформить файл `README.md`, указав:
 - название работы
 - цель работы
 - краткое описание приложения
 - скриншот результата